Restaurante: menú y pedidos

Orden sugerido para presentar a la ingeniera.

Rodolfo · Diego · Miguel · Kendel

Orden	Qué mostrar	Qué explicar
1 · 1 minuto	Aplicación y Docker	Objetivo del sistema, módulos y entorno.
2 · 2 minutos	Diagrama ER, migraciones y modelos	Tablas, llaves foráneas y relaciones Eloquent.
3 · 3 minutos	CRUD de platillos con fotografía	Rutas, MVC, formularios y Storage.
4 · 2 minutos	Validación del servidor	Prueba obligatoria con novalidate y reglas del caso.
5 · 2 minutos	Registrar pedido y abrir ticket	Relación N:M, transacción y cálculo de importes.
6 · 2 minutos	Panel, README y GitHub	Consultas agregadas y evidencias de entrega.

Duración sugerida: 12 minutos; ajustar al tiempo asignado por la docente. Este PDF es apoyo para exponer: la documentación exigida permanece en README y comentarios del código.

PASO 1 · MOSTRAR EL SISTEMA Y SU ARQUITECTURA

Entorno reproducible y separación MVC

En pantalla

```
docker compose ps
```

Abrir **localhost:8000/platillos** y señalar Menú, Nuevo Platillo, Pedidos y Panel.

Qué decir

“Gestionamos el catálogo de platillos y los pedidos de clientes. Laravel procesa las solicitudes; MySQL conserva los datos y Blade genera las páginas.”

Contenedores

- **app:** PHP 8.5 y Laravel 13, puerto 8000.
- **mysql:** MySQL 8, servicio interno `mysql:3306`.
- **phpmyadmin:** consulta de la base desde el puerto 8080.

Los volúmenes conservan MySQL y storage. SQLite en memoria se utiliza únicamente en las pruebas dentro de Docker.

Recorrido de una solicitud

Navegador → **routes/web.php**
→ Form Request, si corresponde
→ **Controlador**
→ **Modelo Eloquent ↔ MySQL**
→ **Vista Blade** o redirección

- **Modelo:** datos, conversiones y relaciones.
- **Vista:** presentación, formularios y mensajes.
- **Controlador:** coordina la operación y responde.

Abrir: `docker-compose.yml`, `routes/web.php` y `resources/views/layouts/app.blade.php`.

La Fase 1 construye la base, modelos y consultas. La Fase 2 completa escritura, pedidos, imágenes, validación y reportes.

PASO 2 · ABRIR DIAGRAMA, MIGRACIONES Y MODELOS

Cuatro tablas y una relación muchos a muchos

Categorías 1 → N **Platillos** 1 → N **Detalle de pedidos** N ← 1 **Pedidos**

Tabla	Campos importantes	Decisión técnica
categorias	nombre, descripción, activo	Catálogo reutilizable para clasificar platillos.
platillos	categoria_id, nombre, precio, tiempo_preparacion, disponible, imagen	Nombre único; dinero en decimal(8,2); imagen como ruta nullable.
pedidos	código, cliente, mesa, fecha, estado, total	Código único; fecha datetime; total decimal(10,2).
detalle_pedidos	pedido_id, platillo_id, cantidad, precio_unitario, subtotal	Conserva qué se pidió, cuánto y a qué precio.

Integridad referencial

`restrictOnDelete` protege categorías con platillos y platillos utilizados en pedidos. `cascadeOnDelete` retira los detalles al borrar su pedido.

La interfaz no ofrece borrar pedidos; la cascada es una regla de la base de datos.

Eloquent

`belongsTo`: platillo → categoría.
`hasMany`: pedido → detalles.
`belongsToMany`: pedido ↔ platillos.

`withPivot('cantidad', 'precio_unitario', 'subtotal')` permite consultar los datos de la tabla intermedia.

PASO 3 · DEMOSTRAR EL CRUD Y LA FOTOGRAFÍA

Seguir un platillo desde el formulario hasta MySQL

Demostración en vivo

1. Crear **Hamburguesa Demo**: categoría Platos fuertes, precio 12.50, tiempo 20, descripción y JPG de menos de 2 MB.
2. Buscarlo y filtrar por categoría. Mostrar la paginación del menú.
3. Abrir su ficha y comprobar los datos y la fotografía.
4. Editar precio a 13.50 y reemplazar la imagen.
5. Cancelar una confirmación de borrado. Para demostrar el borrado definitivo, usar un segundo platillo sin pedidos.

Rutas del recurso

```
Route::resource('platillos',
    PlatilloController::class);
```

index y show consultan; create y edit muestran formularios; store, update y destroy escriben.

Qué explicar en el código

- `StorePlatilloRequest` / `UpdatePlatilloRequest` validan antes de guardar.
- `$request->validated()` entrega los campos validados; `$fillable` limita asignación masiva.
- `Platillo $platillo` usa Route Model Binding: Laravel resuelve el registro por su identificador.
- Formularios con `@csrf` y `@method('PUT')` o `@method('DELETE')`.

Imágenes con Storage

```
$request->file('imagen')
    ->store('platillos', 'public');
```

La base almacena la ruta; el archivo vive en `storage/app/public`. `storage:link` permite servirlo. Al reemplazar se retira el anterior; al borrar, también su archivo. Sin foto se muestra una imagen por defecto.

PASO 4 · PRUEBA OBLIGATORIA DEL DOCUMENTO

Demostrar que el servidor valida los datos

Qué hacer frente a la ingeniera

1. Abrir el formulario de platillos y las herramientas del navegador.
2. Seleccionar el elemento `<form>` y agregar `novalidate`.
3. Enviar datos vacíos o precio 0 y tiempo 181.
4. Mostrar mensajes en español junto a los campos y los valores conservados.
5. Probar un nombre que ya exista. En edición, conservar el propio nombre sí debe permitirse.

Qué decir

“La validación HTML5 mejora la experiencia, pero la regla está en el servidor. Aunque el navegador no valide, Laravel rechaza los datos.”

En Blade: `@error` presenta mensajes y `old()` conserva valores. Después de guardar se redirige con un mensaje flash.

Reglas obligatorias del Caso 5

Regla	Dónde se aplica
Precio mayor que cero	Form Requests de platillos.
Preparación de 1 a 180 minutos	integer y between.
Nombre único	unique en validación y base.
Cantidad entera positiva	StorePedidoRequest.
Platillo disponible	PedidoController, al guardar.
Importes calculados en servidor	PedidoController.

Límites adicionales de esta implementación: precio hasta 999999.99 con dos decimales; cantidades de 1 a 20 y mesas de 1 a 50.

PASO 5 · REGISTRAR UN PEDIDO Y ABRIR SU TICKET

Transacción, detalle y precio histórico

Demostración en vivo

1. Marcar el platillo de demostración como disponible.
2. Crear un pedido con cliente, mesa y solo ese platillo; cantidad **2**.
3. Verificar el total: **13.50 × 2 = 27.00**.
4. Abrir el ticket: cliente, mesa, fecha, líneas, subtotales y total.
5. Cambiar después el precio del menú: el ticket anterior debe conservar su precio unitario.
6. Intentar eliminar ese platillo: queda protegido por su historial de pedidos.

Para mostrar disponibilidad, marcar otro platillo como agotado y comprobar que no aparece en el formulario. El backend vuelve a comprobarlo al guardar.

Qué explicar en PedidoController

1. Validar cliente, mesa, IDs y cantidades.
2. Iniciar `DB::transaction()`.
3. Leer platillos con `lockForUpdate()`.
4. Comprobar disponibilidad y tomar precios de MySQL.
5. Calcular en centavos y guardar pedido y detalles.
6. Confirmar todo; una excepción revierte la transacción.

¿Por qué guardar precio_unitario?
Para mantener el importe cobrado o registrado en ese momento, aunque cambie el catálogo.

¿Por qué no recibir total del formulario?
Porque el cliente podría modificarlo. El servidor calcula cantidad × precio y suma los subtotales.

PASO 6 · EXPLICAR EL PANEL CON CONSULTAS REALES

Agregaciones, relaciones y presentación

Indicador	Cómo se obtiene	Qué significa
Ventas del día	Filtrar fecha y estado; sumar total con sum().	Importe de pedidos del día no cancelados, incluidos los no pagados.
Pedidos por estado	COUNT(*) y groupBy('estado').	Distribución histórica entre pendiente, en preparación, servido, pagado y cancelado.
Platillos más vendidos	withSum() sobre cantidades del detalle, excluyendo pedidos cancelados; ordenar y tomar cinco.	Unidades registradas, no número de filas ni necesariamente ventas cobradas.
Ventas por categoría	Cargar relaciones con with() y sumar subtotales de detalles no cancelados.	Importe histórico agrupado por el catálogo.
Pedidos atendidos	whereIn(['servido', 'pagado'])->count().	Conteo histórico de pedidos en esos estados.

Qué mostrar

Abrir `PanelController.php`, identificar `sum`, `count`, `groupBy`, `withSum` y `with`. Comparar un pedido conocido con su ticket.

Qué aclarar

La fecha se almacena en UTC; vistas y corte diario usan El Salvador. Se permite cambiar cualquier estado. El ticket es imprimible desde el navegador.

CIERRE · EVIDENCIAS QUE PIDE LA RÚBRICA

Terminar con código, README y repositorio

Etiquetas para localizar conceptos

Etiqueta	Archivo o carpeta
[MIGRACION]	database/migrations
[RELACION] / [ENCAPSULAMIENTO]	app/Models
[CRUD-*]	PlatilloController.php
[VALIDACION]	app/Http/Requests
[REGLA-NEGOCIO]	Requests y PedidoController
[SEGURIDAD]	Formularios y controladores
[INYECCION-DEPENDENCIAS]	Parámetros de controladores

Ejemplo de inyección: Laravel proporciona el Form Request al método. Ejemplo de conversión: $casts transforma disponible en boolean y fecha en datetime.

Checklist antes de entrar a clase

- **README:** integrantes, instalación Docker, diagrama ER, reglas, capturas y cambios desde Fase 1.
- **Git/GitHub:** comprobar aportes y fechas reales; Fase 2 exige ≥20 commits nuevos, ≥5 por integrante, ≥3 días, ramas o PR y etiqueta v2.0.
- **Continuidad:** mismo repositorio y referencia de Fase 1 con v1.0. Mostrar evidencia, no solo afirmarlo.
- **Defensa:** llevar las rúbricas impresas y una imagen de menos de 2 MB para la demostración.

Comandos útiles, desde el proyecto

```
docker compose up -d --build --wait
docker compose exec app php artisan route:list
docker compose exec app php artisan test
```

Autenticación y exportación del reporte a PDF/CSV son extras opcionales del documento. Imprimir un ticket no equivale a exportar el reporte.